
Decompose, Then Recompose? A Paired Test of State Interventions at Natural Coding-Agent Stalls

Siddharth Srinivasan

University of California, Berkeley
sidsrinivasan@berkeley.edu

Abstract

Long-horizon coding agents often stall: they keep spending tokens without making progress. A common response is to intervene on the agent’s context or environment: reset the conversation, summarize it, roll back files, or reflect on what went wrong. We ask a narrower question first. If a running agent’s state is decomposed into separable channels, can a *selective recomposition* of those channels, applied at a natural stall, recover a solution, improve the output, or make exploration cheaper than simply letting the agent continue? We record four channels at every turn: conversation history (H), workspace (W), runtime (R) and tool observations (O). Checkpoints pass restore validation before they can be branched. Stalls are detected on unmodified runs with a token-based stall clock and an LLM judge. From each stall we branch into 18 interventions and a matched CONTINUE control. The interventions include a structured, verbatim rendering of the history (a *Ledger*) with ablations of its plans, beliefs, reasoning and current approach. Across 48 stall states from two models on 20 tasks (889 branches, 40k generated tokens each), no intervention beat continuing, which solved 34% of branches. Two history rewrites reliably hurt. Replacing the history with a free-form summary lowered the solve rate by 10 points, 95% CI $[-19, -1]$, and on partial score it was worse than no history at all. Ledger ablations mainly measured how much text was removed. Heavily stripped Ledgers did worse than lightly stripped ones and, on partial score, worse than no history. The most extreme case strips the model’s reasoning, which removes a median 83% of the text, and it lowered the solve rate by 18 points, $[-32, -5]$. Resampling from pre-stall checkpoints, done before the experiment, explains much of the null result. On the 14 states where no pre-stall resample solved the task, no arm solved it either. On stalls the judge typed as under-commitment, continuing already solved 66%. Removing the current approach at its labelled boundary was not reliably better than removing a size-matched random slice. Most arms ran once per state because the run was stopped early to cap spending. We report this as a carefully paired null result and a headroom accounting. We argue that intervention studies should admit stalls only when resampling shows measured headroom, size-match every removal, and include a positive control.

1 Introduction

Coding agents now run for hundreds of turns inside containers, editing repositories and executing tests. Long runs fail in a characteristic way. The agent does not crash; it *stalls*. It keeps generating tokens, re-running the same checks, re-implementing the same idea or hedging instead of committing. The score does not move. Several systems now intervene at such points. They roll the environment back and keep a reflection [26, 29], restart with fresh history when a monitor predicts failure [17], or compact and restructure the context [8, 20]. These systems report improvements on their own

benchmarks. What remains unclear is *which part of the agent’s state* an intervention should change, and whether the stalls in a given run have any room for improvement at all.

We treat a running agent’s state as a composition of separable channels. Using the terminology of our recording contract, these are the conversation history H , the workspace files W , the runtime environment R , and the tool observations O inside the history. A *recomposition* builds a new starting state by keeping, dropping, rolling back or rewriting individual channels, then lets the same worker continue from it. This framing turns “reset”, “rollback” and “summarize” into points in one design space and allows paired comparisons against the natural continuation from the exact same checkpoint. Our research question is:

Can an agent trajectory be decomposed into a state, and can a selective recomposition of that state, applied at a natural stall, recover a solution, improve the output, or make exploration cheaper than letting the agent continue?

This paper reports the first full measurement of that question in our setting (exp48). The result is mostly negative. We think it is informative, for three reasons. First, the comparison is *paired*. Every intervention is compared with CONTINUE from the same validated checkpoint, under the same token budget, with a cluster bootstrap over stall origins. Second, the stalls are *natural*: they were found on unmodified runs by a detector that the worker never sees, not constructed by injecting faults. Third, we measured each stall’s *headroom* before the experiment, independently of its outcome. We resampled the agent from a checkpoint just before the stall and counted how often any resample solved the task. That account explains most of the null result.

Contributions.

- **A state decomposition with validated branching.** We checkpoint H , W and R at every turn, record O provenance, and allow a branch only from checkpoints whose restoration passes exactness checks (§3).
- **A natural-stall pipeline with measured label noise.** A token-based stall clock and a repeated-rejection trigger propose stalls. An LLM judge confirms each stall and assigns a type: *stuck*, *over-commit* or *under-commit*. A hindsight review and a stronger-model shadow judge measure how reliable those labels are.
- **A paired 19-arm measurement on 48 stalls** (889 branches). It covers history resets, free-form summaries, a verbatim structured *Ledger* of the history with label and approach-boundary ablations, workspace rollbacks, runtime discards and observation masking (§4).
- **A headroom accounting**, showing where the null result comes from, and a set of design rules for the next study (§5).

Findings in brief. (i) No arm beats CONTINUE on solve rate or final score. (ii) Replacing the history with a length-matched free-form summary significantly hurts, and on partial score it is worse than giving no history. Heavily stripped structured histories are worse than lightly stripped ones and than none. Stripping the reasoning channel, which removes most of the text, is the most harmful arm. (iii) Discarding the runtime or rolling back the workspace changes nothing measurable. (iv) In 14 of 48 states, neither resamples from before the stall nor any intervention solves the task. In a further 22 states, all judged under-commit, continuing already solves two thirds of branches. (v) Removing the current approach at its labelled boundary is not reliably better than a size-matched random removal. An apparent advantage over an earlier cut is explained by the earlier cut removing more text.

Status of this report. This preprint records a completed experiment, its design, and what it does and does not show. For most arms only one repeat per state completed before we stopped the run to cap spending. We state this in every result. We outline a follow-up design in §5.

2 Related work

Rewind and rollback with retained experience. AgentRewind [29] records aligned checkpoints of context and workspace. It lets the agent itself decide when to rewind and to which checkpoint, and it injects an agent-written memory of the abandoned attempt. With GPT-5.4 and mini-SWE-agent it raises MettleBench success from 62.2% to 87.8%. MettleBench is also one of our two

task sources. RIR [26] frames recovery as choosing when to roll back, where to resume and what reflection to keep, and reports gains on ALFWorld, ScienceWorld and GAIA. GA-Rollback [6] triggers stepwise rollbacks from an assistant model’s review of each action. Graphectory [10] adds diagnostic messages and rollbacks for SWE-bench runs its monitor flags. Reflexion [14] carries verbal reflections across whole trials. These methods *add* information: a reflection, a diagnosis or a memory. Most of them also choose when to act. Our arms instead factor “rollback depth $\times$ retained history” into separate channels, apply them once at an externally detected stall, and add no new advice. Two of AgentRewind’s own ablations are informative here. Without the workspace rewind, success was 43.9%; without the rewind memory, 51.2%. Both are below its continue baseline of 62.2%. Partial recompositions can be worse than continuing, as we find. RIR and AgentRewind appeared while this study was under way.

Restarts, failure prediction and stall detection. Fail-Fast, Restart-Smart [17] restarts a SWE agent with fresh history when a small monitor predicts failure. On SWE-bench Verified this raises one model from 66.6% to 71.8%, while a cold restart reaches 66.8%. Probes on hidden states can predict doomed episodes early [13]. OpenHands [19] ships a rule-based detector for repeated action cycles. Trajectory analyses find that coding-agent failures are often epistemic, begin early and become visible only late [28]. They also find that failed runs often locate the right files [11]. Both observations fit our finding that many late stalls are ceilings.

What the history should contain. Handoff Debt [3] hands interrupted SWE-bench tasks to fresh successors with no context, raw traces, summaries or structured notes. It finds large cost savings but small, model-dependent effects on solve rate. In Recovery-Bench [16, 5], summaries of a failed attempt did worse than the environment alone, and the full failed history did worst. Observation masking matches LLM summarization on solve rate [8]. A controlled harness study finds that context management helps mainly by preventing overflow [2]. Concurrent work prunes historical reasoning blocks during a run with a small reward gain [18]. Other work shows that plans must stay in context to keep influencing behaviour, and that reasoning traces can re-derive them [12]. In our one-time interventions at a stall, stripping the reasoning channel was the most damaging manipulation. However, it also removed most of the packet, and our design cannot separate content from amount. MemArbiter [1] decomposes history into functional memory banks for ALFWorld under tight budgets. Wang et al. [20] also use the name “Ledger”, for a runtime layer that distills interactions into an explicit execution state and blocks redundant actions at every step. Our Ledger is unrelated. It is a one-time, verbatim, structured rendering of the worker’s own history at the stall, with no action gating.

Why resetting is expected to help. Models condition on their own earlier errors [15]. They get lost after early wrong turns in multi-turn conversation [4]. They use information in the middle of long contexts poorly [9]. A formal model predicts that a clean restart dominates a retry in a contaminated context [25]. Our reset arms test this prediction directly at natural stalls, and do not confirm it within a 40k-token budget.

Evaluating recovery. Agreement on the best action across runs cannot distinguish high-success from near-zero-success regimes. Recovery studies should therefore report the fraction of checkpoints where every option scores zero [22]. In our data that fraction is 29% of states (the ceiling class). A saved state is not automatically a valid resume point [27]. Rollback restores only the state it controls [21]. This is why we validate each restore and report R separately.

3 Setup

3.1 Agent, models and tasks

Harness. Workers run in mini-SWE-agent 2.4.6 [7, 23], wrapped by a recording layer. The agent has one bash tool, executed in the task container; each command runs in a fresh subshell. Observations of 10,000 characters or more are delivered as their first and last 5,000 characters. The agent ends an attempt by issuing a fixed submit command. Natural runs have only runaway guards (\$3, 500 model calls, 6 hours), and none of the 82 natural runs in our pool hit one.

Models. We use two low-cost reasoning models through OpenRouter, each pinned to one provider with fallbacks disabled, at temperature 1 and maximum reasoning effort. GPT-6 Luna was served by OpenAI with detailed reasoning summaries. DeepSeek V4.1 Flash was served in fp8 by GMICloud. Every branch uses the same model and recorded settings as the run it branches from.

Tasks. States come from 20 tasks in two sources. *MettleBench* [29] rewrites Terminal-Bench, ProgramBench, SWE-bench, ProjectEval and GitTaskBench tasks into ordered checklists of acceptance criteria. A submission receives only the first unmet criterion as feedback, and the run ends after five consecutive submissions fail on the same criterion. Our score is the fraction of criteria that pass; a full solve requires all of them. *SWE-smith long* [24] uses multi-bug SWE-smith instances with 8–60 failing tests, from five Python projects. The failing tests are hidden. The score is the fraction of those tests passing, a full solve is the standard “resolved” criterion, and a submission ends the run. Grading always happens privately, on an isolated copy of the captured workspace, and grades never reach the worker.

3.2 State recording and validated branching

We record four channels:

- **History H:** the exact message list delivered to the model, with counters and the execution cursor.
- **Workspace W:** a tar of `/workspace` with a manifest giving each entry’s type, mode, owner, size, modification time and SHA-256.
- **Runtime R:** filesystem changes outside `/workspace` relative to the image (`docker diff`), archived with the process table. Processes are recorded but never restored.
- **Observations O:** references to the exact tool-result bytes delivered inside H.

A checkpoint is captured before the first call, after every step that changes **W**, at every detector grade request, at every submission, and at the end of the run. A checkpoint is *branchable* only if it has no pending actions, its runtime is supported, and its history round-trips through the store. Restoring a branch must reproduce the **W** and **R** manifests exactly. It must also pass history checks: the messages, cursor, task, inherited counters and delivered observations must match exactly. Any failure aborts the branch. A failed capture never yields a branchable checkpoint. All 127 resamples (§3.4) passed **W**, **R** and **H** restore validation.

3.3 Natural stall detection

Stall clock. *Progress* is a valid private grade that strictly improves on the best so far in the current scope, by score, number of passing checks, or number of executing checks. The detector (v8.1) proposes a stall at a graded, branchable checkpoint when four conditions hold: at least 10,000 generated tokens have passed since the last progress, at least 5 model turns have passed, the grade is valid, and the task is unsolved. When the clock condition holds but no grade was taken, the detector requests a private grade, at most once every 5 turns. A second trigger fires when a native submission is rejected on the same first unmet criterion as the previous one with no progress in between. Warnings can recur while the agent remains stalled. The worker never sees the detector.

Judge. Each warning is shown to an LLM judge (GLM-5.3 Flash, maximum reasoning effort). The judge sees a prefix-only, untruncated record: the task, each turn’s reasoning, text and commands, the observations as delivered, the feedback and the private grades, with model identity masked. It votes three times, and a warning is confirmed on a strict majority for “stall”. The first confirmed warning in a run is its primary stall. A separate call, which also sees the task’s reference fix, assigns a type:

- *stuck*: nothing works, and the agent is going round in circles;
- *over-commit*: the agent believes a wrong or incomplete fix is done;
- *under-commit*: the agent already has the right fix but will not apply or finish it;
- *unclear*.

Types are used only for analysis. The originals were typed by hindsight review (below), and the resampled states by this judge call. In a provisional, in-sample validation against hindsight labels

on 71 runs, the detector plus judge recalled 21 of 22 genuine stall episodes at 0.70 precision. Judge-hindsight agreement was $\kappa = 0.50$ for stall calls and $\kappa = 0.46$ for types. Hindsight labels were written by Claude reviewers with full-run access under a conservative rubric. We have no human agreement figures.

3.4 States and headroom resampling

Originals. The originals are the first judge-confirmed warning in each of 30 development runs, plus one confirmed stall from a later harvest run. Six were excluded because their checkpoints existed only in an offline replay of the detector and could not be branched, which left 25. These originals were selected with a single judge vote and draft-era prompts. The frozen three-vote rule described above came later.

Resampling. For each original we defined the *origin*: the latest branchable checkpoint at or before the last progress point preceding the stall. From the origin we ran five natural continuations (seven for one origin). These used native history, the live v8.1 detector, a 60k generated-token cap, and a stop at the first full solve. In 12 origins there was no progress point before the stall, so the origin is the start of the run and the continuations are fresh reruns. The resamples yielded 127 runs, 58 full solves and 92 judge-confirmed branchable stalls, for \$9.91 in total (\$6.25 worker, \$3.66 judge). For each origin we then picked one confirmed resampled stall uniformly at random, with a hashed seed, before any branch outcome existed. 23 origins had one, for 48 states in 25 origin clusters. The resample outcomes also define the recoverability classes of §4.2; those classes were computed afterwards but depend on no exp48 outcome.

3.5 Interventions

Every branch starts from the stall checkpoint’s restored state. Arms are written as channel assignments; unspecified channels are kept as they were at the stall. A *fresh* history contains only the system prompt, the fixed task statement and the sentence “The workspace may contain prior work. Inspect it as needed.” A *packet* (Ledger view or summary) is appended to a fresh history. No arm adds advice or diagnoses.

Channel arms. CONTINUE restores *H*, *W* and *R*. Reset *H* uses a fresh history. Fresh *R* discards the runtime and keeps *H*. Roll back *W* restores the workspace (and its runtime) from the *donor*: the highest-scoring branchable checkpoint before the stall, which in all 48 states is the last progress point. The rollback is used either with native *H* or with a fresh history. Mask *O* replaces tool-result contents after the last progress point with a fixed elision marker, keeping the tool calls. It could be built only where the recorded delivery evidence was unambiguous (21 states).

Summary. The state’s own worker model, with its recorded settings, writes a prose summary of the worker-visible transcript through the stall using a frozen prompt that forbids advice. Its length is matched to the full Ledger view within $\pm 15\%$, with at most three attempts. It could not be built for 2 states.

Ledger. The Ledger is a verbatim, structured rendering of the worker’s own history.

- *Items.* The items are worker sentences (visible text, reasoning, provider reasoning summaries and code comments) and tool outputs. Nothing is paraphrased.
- *Labels.* An LLM sentence classifier (Jev 1.13) labels every sentence on four independent axes: epistemic status, action, self-assessment and content. Tool outputs receive deterministic labels. A second LLM (GLM-5.3 Flash) records explicit back-references between items.
- *Approach windows.* An *implemented-decision window* is a stretch of reasoning ending in a decision the worker then carried out, such as a workspace change or a submission. It is our operational unit of “approach”. The *current approach* is the last window before the stall.
- *Packets.* Items are packed newest-first into a 3,000-token packet, with at least half the budget reserved for the current window, under a header stating that the notes are verbatim and may describe code that has since changed. The full view has a median of 2,982 tokens.

All removal views are cut from this packet and *not refilled*, so they are also shorter than the full view:

- *–plans* drops sentences whose action label is a proposal, decision or next step. A median of 63% of the packet remains.
- *–beliefs* drops sentences labelled as hypotheses or inferences (57% remains).
- *–plans –beliefs* drops both (39% remains).
- *–reasoning channel* drops every sentence from the worker’s reasoning or the provider’s reasoning summaries (17% remains).

The approach-boundary views are defined only where the stall has at least two windows (33 states):

- *–current approach* removes the current window’s sentences and the outputs of its commands.
- The *random matched slice* removes a token-matched ($\pm 10\%$), stratified random set of other items.
- The *shifted cut* moves the removal boundary earlier by 37.5% of the window, to the nearest turn boundary.
- The *tombstone* variant removes the current approach but keeps its anchor decision sentence and its last failing tool output.
- Two further arms combine the full view, or *–current approach*, with rolling *W* back to the last branchable checkpoint before the first removed item. Together with the full view and *–current approach*, these form a 2×2 of *idea* removal (H) by *code* removal (W).

Matched random controls for the plans and beliefs ablations were declared before any Ledger outcome. The pre-declared trigger required both repeats of every arm to finish, which did not happen, so they were not run.

3.6 Budget, outcomes and analysis

Each branch may generate up to 40,000 tokens, including reasoning. *Best* is the highest valid score over the branch’s checkpoints. *Solved* means some checkpoint was a full solve. Values at smaller budgets are read off the branch’s trace and carried forward after it stops. We planned two repeats of all 19 arms on all 48 states (1,824 branches), with all first repeats running first. We stopped the run to cap spending while the second repeat was in progress. At that point, of the 912 first-repeat cells, 784 had finished, 92 were unavailable under the rules above, 35 had failed to build, and 1 had failed on infrastructure. Second repeats had finished for CONTINUE on all 48 states, for Reset H on 13 and for the full Ledger on 14. The total is 889 recorded branches. Known spend was \$28.57, and total API credit use was \$30.46. Per-branch dollar figures are lower bounds, because some provider operations are unpriced. Ledger and summary preparation costs are not allocated to branches.

For each state and arm we average over completed repeats. We then compare arms by paired differences over the states where both have a result. 95% intervals come from a percentile bootstrap that resamples the 25 origin clusters (an original and its resampled state form one cluster) 2,000 times. Factorial effects are computed per state and bootstrapped in the same way.

4 Results

Unless noted otherwise, differences are paired per state against CONTINUE. Each arm’s per-state value is the mean over its completed repeats. 95% intervals come from a cluster bootstrap over the 25 stall origins (2,000 resamples, percentile). “Solve” means the hidden evaluation passes in full; “best” is the highest partial score reached within the budget. We did not correct for multiple comparisons. With 18 arms and two metrics, a few intervals that exclude zero are expected by chance. We therefore emphasise effects that agree in direction across metrics and subsets.

4.1 No intervention beats continuing

Table 1 and Figure 1 show all 18 interventions. None has a positive paired difference against CONTINUE whose interval excludes zero, on solve rate or on best score. Point estimates are negative for 17 of 18 arms on solve rate. The one exception is the observation mask, which could be built on only 21 states, mostly easy ones. The absolute numbers look better than the paired ones for a structural reason. Arms restricted to multi-approach states (the [†] block) have lower raw solve rates because

Table 1: **All arms on all 48 stall states.** Br. is the number of completed branches. Solve and Best are per-state means (repeats averaged first), which is the scale of the paired analysis. CONTINUE completed two repeats per state; all other arms completed about one, and RESET H and LEDGER (FULL) 1.3. Δ columns are paired differences against CONTINUE with 95% cluster-bootstrap intervals. **Bold:** the interval excludes zero. Arms marked [†] exist only where the stall has at least two labelled approach windows (33 states, 29 for **W** rollback). [‡] The observation mask could be built on only 21 states, 19 of them under-commit, so its raw solve rate is not comparable to the other rows.

Arm	Br.	States	Solve	Best	Δ solve	Δ best
CONTINUE (native H, W, R)	96	48	.344	.798	–	–
<i>Channel resets and rollbacks</i>						
Reset H (task prompt only)	61	48	.271	.781	–.073 [–.15, –.01]	–.017 [–.06, .02]
Fresh R, keep H	50	48	.312	.804	–.031 [–.08, .02]	+.006 [–.03, .04]
Reset H + fresh R	50	48	.250	.759	–.094 [–.19, .00]	–.040 [–.11, .02]
Roll back W, keep H	50	48	.312	.792	–.031 [–.14, .08]	–.006 [–.05, .04]
Roll back W + reset H	50	48	.271	.773	–.073 [–.20, .05]	–.025 [–.09, .04]
Mask tool outputs (O) [‡]	21	21	.619	.792	+.048 [–.06, .16]	–.014 [–.09, .07]
<i>History rewrites</i>						
Summary H	48	46	.261	.718	–.098 [–.19, –.01]	–.090 [–.17, –.02]
Ledger H (full)	62	48	.292	.729	–.052 [–.14, .02]	–.069 [–.16, .01]
– plans	50	48	.292	.788	–.052 [–.14, .03]	–.010 [–.05, .03]
– beliefs	50	48	.250	.754	–.094 [–.19, –.01]	–.045 [–.10, .01]
– plans – beliefs	50	48	.229	.736	–.115 [–.23, –.01]	–.062 [–.14, .00]
– reasoning channel	50	48	.167	.698	–.177 [–.32, –.05]	–.100 [–.19, –.02]
<i>Approach-boundary removals from the Ledger[†]</i>						
– current approach	35	33	.152	.781	–.045 [–.12, .03]	+.001 [–.02, .03]
– current, with tombstone	34	32	.125	.774	–.047 [–.13, .03]	+.001 [–.02, .03]
– random matched slice	35	33	.121	.762	–.076 [–.16, –.02]	–.018 [–.05, .01]
– cut shifted earlier	35	33	.091	.699	–.106 [–.22, –.01]	–.081 [–.18, –.01]
full + W rollback of current	31	29	.172	.732	–.052 [–.13, .03]	–.052 [–.14, .00]
– current + W rollback	31	29	.138	.757	–.086 [–.20, .00]	–.027 [–.08, .01]

those states are harder, not because the interventions are worse. For this reason every comparison in this paper is paired.

Interventions also did not make exploration cheaper. CONTINUE cost \$0.033 per branch and \$0.097 per solve (known worker and grading cost, 19.3k generated tokens per branch). Every rewritten history used more generated tokens per branch (22.6–28.6k) and cost more per solve (\$0.11–\$0.32). The Ledger’s preparation cost is shared and not included. The cost curves in Figure 2 show that CONTINUE leads at nearly every budget. Reset and rewrite arms lose most of their deficit early: at 10k tokens the Ledger solved 5% of branches against 15% for CONTINUE. Rebuilding context costs tokens before any new progress is made.

4.2 Where the null comes from: headroom

Before exp48, and without looking at any intervention outcomes, we resampled each stall’s run five times (seven for one origin). The resamples started from the last progress point before the stall, or from the start of the run when there was none (§3.4). This gives an outcome-independent *recoverability* class for every state:

- **Ceiling** (14 states, 7 origins). The stall comes late, and no resample from before it solved the task. Mean start score 0.71.
- **Recoverable** (11 states, 6 origins). The stall comes late, and at least one resample from before it solved the task.
- **Early origin** (23 states, 12 origins). There was no progress point before the stall, so the resamples were fresh reruns. For 11 of these 12 origins, at least one fresh rerun solved the task. Mean start score 0.08.

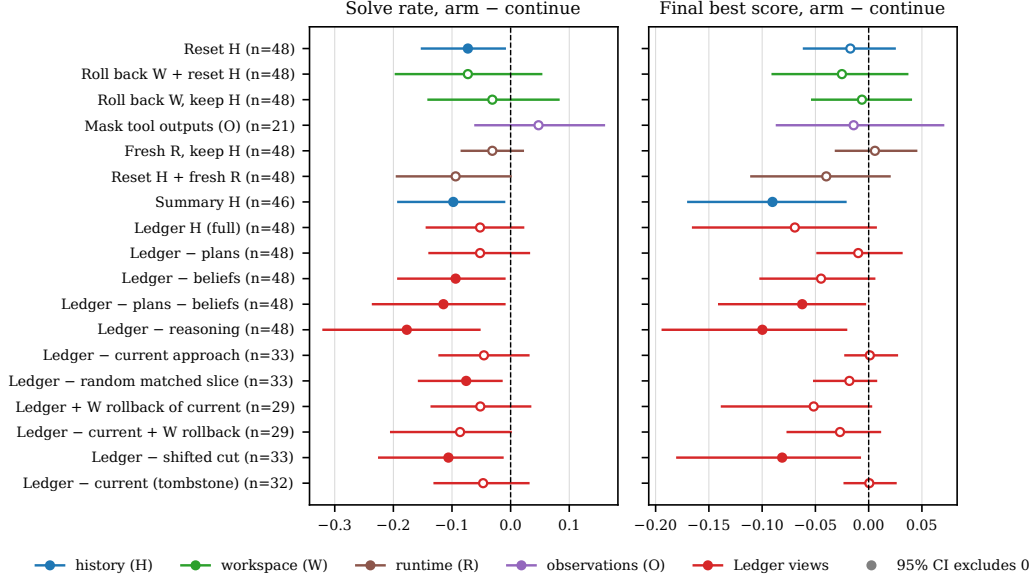


Figure 1: **Paired differences against CONTINUE** (left: solve rate; right: best score), with 95% cluster-bootstrap intervals over stall origins. Filled markers: interval excludes zero. No arm is reliably better than continuing. Summary and removal of the reasoning channel are reliably worse on both metrics.

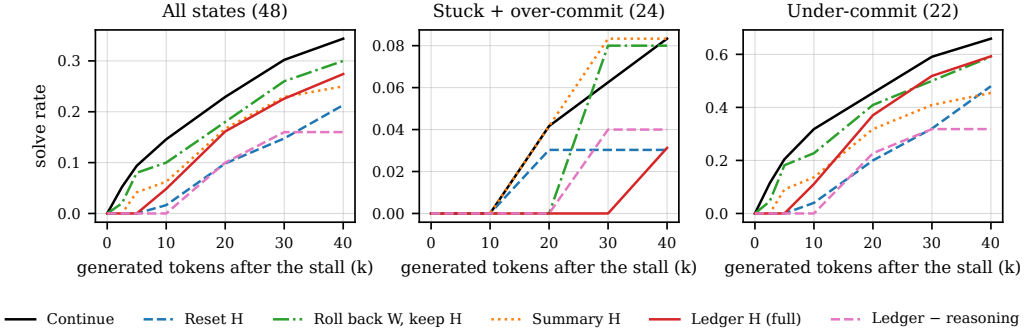


Figure 2: **Solve rate as a function of generated tokens after the stall**, carried forward after a branch ends. Left: all states. Middle: states judged stuck or over-commit (the primary population). Right: under-commit. Among the plotted arms, continuing leads or ties at every budget, except at 30k tokens in the middle panel, where two arms are ahead by about two points. In that panel all rates are below 10%.

Figure 3 shows the consequence. On the 14 *ceiling* states no branch in any arm reached a full solve. This is the pre-experiment resample prediction, now confirmed across every intervention. Gains in partial score there were small, at most +0.026 best (Reset H, $[0.00, 0.08]$). The 23 *early-origin* states are a different problem. The agent had made no measurable progress before the stall (mean score 0.08), yet 20 of these states are typed under-commit, meaning the agent already held the right fix. CONTINUE solved 59% of these branches, and each of the nine main interventions we broke down by class was lower. On the 11 *recoverable* states, the population the question is actually about, CONTINUE solved 27%, rolling back W with or without a history reset also solved 27%, and Ledger without reasoning solved 0% ($-0.27, [-0.59, -0.04]$). With 11 states and one repeat per arm, solve-rate intervals on this population are up to 90 points wide.

The judge’s stall type splits the states in a similar way (Figure 2, middle and right). On the 24 states typed *stuck* or *over-commit*, CONTINUE solved 8% of branches, and every arm was within

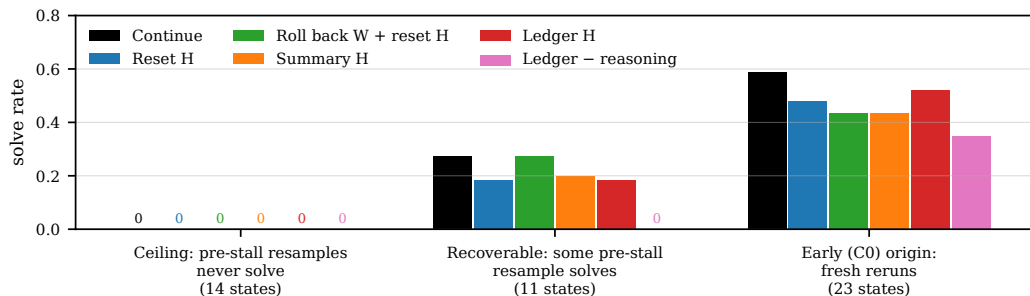


Figure 3: **Solve rate by recoverability class**, assigned from pre-experiment resamples. No arm solves any ceiling state. On recoverable states, removing the reasoning channel from the Ledger never solves. On early-origin states, continuing is already strong.

9 points of it on solve rate and within 0.04 on best score. On the 22 states typed *under-commit*, CONTINUE solved 66%. Summary (-0.20 , $[-0.36, -0.04]$) and Ledger without reasoning (-0.34 , $[-0.57, -0.11]$) were significantly worse. Stall type, model, benchmark and recoverability class are strongly confounded. Of the 22 under-commit states, 19 come from DeepSeek and 20 have early origins, most of them on SWE-smith tasks. Of the 14 ceiling states, 10 are GPT-6 Luna over-commit stalls on MettleBench tasks. We therefore do not interpret type, model and benchmark effects separately.

In short, about half of the states had no headroom for any intervention to show, and in the other half, continuing was already the strongest policy.

4.3 What the history carries

Although no arm helps, the arms that hurt are informative (Table 1, middle block).

Summaries lose information that matters. Replacing H with a free-form summary, matched in length to the 3k-token Ledger, lowered both metrics against CONTINUE. It also lowered the best score against a plain history reset (-0.072 , $[-0.15, -0.01]$). A summary is not merely less helpful than the full history; on best score it is worse than no history at all. The verbatim Ledger was not significantly different from the summary ($+0.014$ best, $+0.043$ solve).

Ledger ablations mostly measure how much text is removed. Removal views are cut from the 3k-token packet and are not refilled, so every label ablation also shrinks the packet (§3.5). Figure 4 plots each Ledger arm’s paired effect against the median share of the packet it keeps. The heavily stripped packets, which keep 41% or less, are all worse than continuing: solve-rate differences range from -0.11 to -0.18 . The lightly stripped ones, which keep 56% or more, range from -0.05 to -0.09 . The most harmful arm removes the reasoning channel. Because most worker prose is reasoning or provider reasoning summaries, that arm keeps a median of only 17% of the packet, mostly tool outputs and short visible messages. It was worse than continuing (-0.177 solve, $[-0.32, -0.05]$) and worse than the full Ledger (-0.125 , $[-0.24, -0.02]$). It was also worse than no packet at all on best score (-0.083 , $[-0.15, -0.03]$), and it was the only arm with no solves on the recoverable states. A heavily stripped history thus seems to hurt more than an empty one, perhaps because observations without the reasoning that connected them mislead the worker. We cannot tell whether reasoning matters because of its content or because it is most of the text. The size-matched controls declared for the label ablations were not run, and removing plans or beliefs is confounded with size in the same way. We therefore draw no conclusion about which *kind* of content carries the state.

Runtime and workspace are not where stalls live. Discarding the runtime while keeping the history was indistinguishable from CONTINUE ($+0.006$ best, $[-0.03, 0.04]$). It also looked cheaper, but not reliably. On first repeats, where both arms ran in the same pass, it cost about 11% less per branch (paired $-\$0.0035$, $[-0.0076, 0.0010]$). Pooling CONTINUE’s second repeats, which ran later and cost more, would inflate this to 17%. Rolling W back to an earlier checkpoint, whether the history was kept or reset, changed nothing measurable. In the idea $\times$ code factorial on the 29

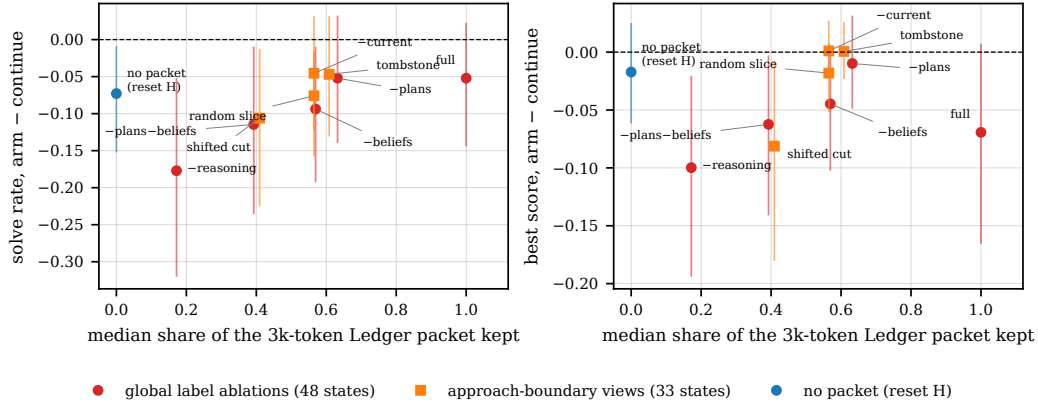


Figure 4: **Paired effect of each Ledger view against how much of the 3k-token packet it keeps** (median over the view’s states). Bars are 95% cluster-bootstrap intervals. Heavily stripped packets are worse than lightly stripped ones and, on best score, worse than no packet. The approach-boundary controls (squares) differ in size as well: the shifted cut keeps less than the boundary removal, and the random slice is token-matched to it.

multi-approach states, which crosses removing the current approach from H with rolling back its W changes, neither main effect nor the interaction differed from zero.

4.4 Approach boundaries: no reliable signal

The approach-boundary arms test whether the *current approach* is a meaningful unit of state. Here the current approach is the labelled window of implemented decisions that the agent is pursuing at the stall. Removing it did not help against CONTINUE. Two controls compare it with other removals:

- **Boundary vs random matched slice.** This is the size-matched control; for 8 of 33 states only an approximate match within 10% was found. Removing the current approach instead of a same-size random selection of other items gave $+0.019$ best $[-0.02, 0.07]$ and $+0.030$ solve $[0.00, 0.10]$, with 1 state better and none worse on solve.
- **Boundary vs shifted cut.** Moving the cut 37.5% of the window earlier did worse than the exact boundary: $+0.082$ best $[0.00, 0.20]$ in favour of the boundary, with 4 states better and 0 worse. But the shifted cut also removes more text (median 41% of the packet kept, against 56%). That difference alone would produce this contrast (Figure 4).

Keeping the removed approach’s anchor decision and last failing output (the *tombstone* variant) made no difference (-0.001 best). In short, we find no evidence that our labelled approach windows are special units of state beyond their size. A fair test needs size-matched controls for every removal, more repeats, and approach labels we trust. Our windows come from LLM sentence labels and have not been checked by humans.

4.5 Label reliability

The primary judge selected the states; its labels were frozen before exp48. Two record-only second opinions let us check that selection. They are a hindsight review of the originals and a stronger-model shadow judge at the same gates for the resampled states. The hindsight review called 15 of the 24 reviewed originals genuine stalls and contested 9; one original went unreviewed. The shadow judge agreed on 19 of the 23 resampled states and contested 4. In total, the second opinion agreed on 34 of 47 reviewed states. Contested states were easier: CONTINUE solved 58% of their branches, against 26% on agreed states. On the 15 original states whose hindsight review called the stall genuine, CONTINUE solved 27% of branches. No intervention exceeded that except the observation mask, which could be built on only 6 of them. Label noise is therefore substantial, and it pushes the sample toward states with little headroom. We return to this in §5.

5 Discussion

What exp48 shows. At natural stalls of two current models, selectively recomposing H, W, R and O did not beat letting the agent continue. That is the main result, and it holds on both metrics and in every population we examined. It does not show that recomposition cannot help. The recoverable population, the only one where an intervention could plausibly beat continuing, had 11 states and mostly one repeat per arm. Under those conditions, only a very large effect could have been detected.

The experiment does support three narrower claims. First, *rewritten histories cost more than they return*. A length-matched free-form summary is worse than no history at all on partial score. Heavily stripped structured histories are worse than both lightly stripped ones and no history. Every rewritten history spends more tokens than continuing before it makes progress. This argues against intervention designs that begin by summarizing or pruning. Second, *runtime and workspace state are not where these stalls live*. Discarding R or rolling back W is neutral, so interventions can be cheap on those channels. Third, *our removal ablations were confounded with size*, and this is a methodological lesson. Without refilling, every label or boundary removal also shortens the packet, and the one size-matched boundary control showed no reliable effect. Whether particular kinds of content carry the state remains open. Concurrent work suggests reasoning can be pruned during a run [18], while plans need active management to persist [12]. Our design cannot adjudicate between these.

Why the design missed. The resample stage told us, before the experiment, which states had headroom, but we did not use it to admit states. Selection by an LLM judge produced three kinds of states. *Ceiling* states (29%) were ones that no resample and no intervention solves. *Early-origin* states (48%) were ones where the agent had made no measurable progress yet mostly already held the right fix, and continuation already works well. That left 23% of states where the comparison is informative. The judge’s stall type did not rescue the design either: 14 of the 24 *stuck* or *over-commit* states were ceiling states, and 20 of the 22 *under-commit* states were early-origin states. We also measured substantial label noise. Of the 25 originals, the hindsight review contested 9.

Design rules for the follow-up. We propose the following changes for a follow-up study:

1. **Admit by measured headroom.** Keep only stalls where pre-stall resamples solve some of the time but continuation from the stall rarely does. Draw tasks from a band where the model solves 10–50% of fresh attempts.
2. **Include a positive control.** An arm with privileged information should recover a clear majority of admitted stalls. If it does not, the admitted stalls have no headroom and the arms cannot be compared.
3. **Fewer arms, more repeats.** Five arms (continue, reset H, Ledger, Ledger without the current approach, and its matched control) with at least five repeats per state, instead of 19 arms with about one.
4. **Hand-labelled stalls and approaches.** Replace LLM approach labels with human labels on the admitted stalls, and report agreement.
5. **Size-match every removal.** Refill removed packet space from the same history, or pair every ablation with a token-matched random removal, so that content and amount are not confounded.

Limitations. *Repeats:* only CONTINUE completed two repeats per state. Most other arms completed one, because we stopped the run to cap spending before the second repeat finished (789 queued branches were not run). *Scale:* 48 states from 25 origins on 20 tasks and two low-cost models. The resampled states are not independent of their origins, which the cluster bootstrap accounts for only in part. *Multiplicity:* 18 arms, two metrics and several subsets, with no correction; the intervals that exclude zero should be read as descriptive. *Labels:* stall detection, stall type, approach windows and Ledger fields are all produced by LLMs, and none was checked against human labels. *Size:* removal views shrink the packet, and matched controls exist only for the current-approach removal. *Availability:* the six approach-boundary arms were undefined on the 15 states with a single approach window. They were recorded as equivalent to a history reset, or as removing nothing, and were not run. The observation mask could be built on only 21 states. *Runtime:* R restoration does not restore running processes. *Budget:* a fixed budget of 40k generated tokens may be too short for a restarted agent to pay back the cost of rebuilding its context. *Confounding:* stall type, model, benchmark and recoverability class are strongly entangled in this sample.

Conclusion. Decomposing an agent’s trajectory into state channels, and branching from validated checkpoints into selective recompositions, is practical. Our harness ran 889 paired branches from 48 natural stalls. At the stalls this pipeline selected, no recomposition beat continuing. Rewriting the history costs more than it returns, and stripping it heavily is worse than dropping it. Most of the null result is explained by headroom that could be measured before the experiment. The question stays open. The next study should admit stalls by resampled headroom and include a positive control.

References

- [1] Jiajun Dong, Yutao Hu, Fengrui Fan, Shihan Dou, Yueming Wu, and Deqing Zou. MemArbiter: Decision-Time Memory Arbitration for Long-Horizon LLM Agents, August 2026. URL <https://arxiv.org/abs/2608.02113>.
- [2] Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An Empirical Study of Harness Design for Coding Agents, September 2026. URL <https://arxiv.org/abs/2609.20804>.
- [3] Dipesh KC and Anjila Budathoki. Handoff Debt: The Rediscovery Cost When Coding Agents Take Over Interrupted Tasks, June 2026. URL <https://arxiv.org/abs/2606.02875>.
- [4] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs Get Lost In Multi-Turn Conversation, May 2025. URL <https://arxiv.org/abs/2505.06120>.
- [5] Letta. Introducing Recovery-Bench: Evaluating LLMs’ Ability to Recover from Mistakes. Letta Research Blog, August 2025. URL <https://www.letta.com/blog/recovery-bench/>.
- [6] Xingzuo Li, Kehai Chen, Yunfei Long, Xuefeng Bai, Yong Xu, and Min Zhang. Generator-Assistant Stepwise Rollback Framework for Large Language Model Agent, March 2025. URL <https://arxiv.org/abs/2503.02519>. EMNLP 2025 Main.
- [7] Kilian Lieret and Carlos E. Jimenez. mini-SWE-agent: The Minimal AI Software Engineering Agent. Software, version 2.4.6, July 2026. URL <https://github.com/SWE-agent/mini-swe-agent>.
- [8] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management, August 2025. URL <https://arxiv.org/abs/2508.21433>. 4th Deep Learning for Code (DL4C) Workshop at NeurIPS 2025.
- [9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts, July 2023. URL <https://arxiv.org/abs/2307.03172>. Accepted for publication in Transactions of the Association for Computational Linguistics (TACL).
- [10] Shuyang Liu, Yang Chen, Rahul Krishna, Saurabh Sinha, Jatin Ganhotra, and Reyhan Jabbarvand. Process-Centric Analysis of Agentic Software Systems, December 2025. URL <https://arxiv.org/abs/2512.02393>.
- [11] Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories, October 2025. URL <https://arxiv.org/abs/2511.00197>.
- [12] Aman Mehta and Anupam Datta. Plans Don’t Persist: Why Context Management Is Load Bearing for LLM Agents, June 2026. URL <https://arxiv.org/abs/2606.22953>.
- [13] Kai Ruan, Zihe Huang, Ziqi Zhou, Qianshan Wei, Jinghao Lin, Xuan Wang, and Hao Sun. Doomed from the Start: Early Abort of LLM Agent Episodes via a Recall-Controlled Probe Cascade, July 2026. URL <https://arxiv.org/abs/2607.06503>.
- [14] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, March 2023. URL <https://arxiv.org/abs/2303.11366>.

- [15] Akshit Sinha, Arvinth Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping. The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs, September 2025. URL <https://arxiv.org/abs/2509.09677>. Published at ICLR 2026.
- [16] Shangyin Tan, Kevin Lin, Koushik Sen, and Matei Zaharia. Recovery-Bench: Evaluating Agentic Recovery from Mistakes. NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle, 2025. URL <https://openreview.net/forum?id=8FZRnDgDxq>.
- [17] Chenyu Wang, Yunbo Lyu, Junda He, Zhou Yang, Chenxing Zhong, Yaniv Harel, and David Lo. Fail-Fast, Restart-Smart: Early Failure Prediction and Restart for SWE Agentic Tasks, August 2026. URL <https://arxiv.org/abs/2608.03222>.
- [18] Mingxuan Wang, Fei Luo, Bo Wang, Guorun Yao, Yinglong Guo, Chao Ning, Hongyue Chen, Yanbiao Ma, and Jungong Han. When Can Agents Forget Their Reasoning? ICLR for Long-Horizon Agent Context Compression, September 2026. URL <https://arxiv.org/abs/2609.29875>.
- [19] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents, July 2024. URL <https://arxiv.org/abs/2407.16741>. Published at ICLR 2025.
- [20] Zehao Wang, Yisen Xu, Chenglin Li, Chao Peng, Bram Adams, Ahmed E. Hassan, and Tse-Hsun (Peter) Chen. Turning Interaction History into Execution State: A Runtime Layer for Long-Horizon Coding Agents, August 2026. URL <https://arxiv.org/abs/2608.00808>.
- [21] Guanlong Wu, Dahui Li, Ke Jiang, Jianyu Niu, Cong Wang, and Yinqian Zhang. Safe to Resume? Breaking Execution Continuity of Agent Execution via Rollback, August 2026. URL <https://arxiv.org/abs/2608.29381>.
- [22] Dong Xu, Zhangfan Yang, Jiantao Wu, Shipeng Zhang, Zexuan Zhu, Jiangqiang Li, Jun Zhang, and Junkai Ji. Same Winners, Different Success Rates: Evaluating How LLM Agents Recover from Failures, September 2026. URL <https://arxiv.org/abs/2609.34215>.
- [23] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, May 2024. URL <https://arxiv.org/abs/2405.15793>. Published at NeurIPS 2024.
- [24] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling Data for Software Engineering Agents, April 2025. URL <https://arxiv.org/abs/2504.21798>.
- [25] Zhanfu Yang. Why Retrying Fails: Context Contamination in LLM Agent Pipelines, May 2026. URL <https://arxiv.org/abs/2605.08563>.
- [26] Yi Yu, Liuyi Yao, Yaliang Li, Enshu Wang, and Libing Wu. Rollback the World, Keep the Reflection: Rollback-Induced Reflection for Long-Horizon LLM Agents, September 2026. URL <https://arxiv.org/abs/2609.18304>.
- [27] Zhihui Zhang and Wei Liu. Recoverability as a System Primitive for Long-Horizon AI Agents, September 2026. URL <https://arxiv.org/abs/2609.13672>.
- [28] Xiangxin Zhao, Han Li, Shuaiting Li, Tianyi Zhao, Earl T. Barr, Federica Sarro, and He Ye. Failure as a Process: An Anatomy of CLI Coding Agent Trajectories, July 2026. URL <https://arxiv.org/abs/2607.09510>.
- [29] Yu Zhuang, Kefei Chen, Yitong Duan, Shuxin Zheng, Jian Li, and Xu-Yao Zhang. AgentRewind: Recoverable Execution for Long-Horizon LLM Agents, August 2026. URL <https://arxiv.org/abs/2608.14380>.

A Population breakdowns

Table 2 gives, for the ten arms defined on all 48 states, the per-state solve rate and the paired solve-rate difference against CONTINUE in each population. These breakdowns use the same analysis as the main paired results but a different bootstrap seed, so their intervals differ slightly in the last digit. The populations overlap and are strongly confounded (§4.2).

Table 2: **Solve rate by population for the ten arms defined on all states.** Each cell gives the per-state solve rate, then the paired solve-rate difference against CONTINUE with a 95% cluster-bootstrap interval. **Bold:** the interval excludes zero. Recoverability classes come from the pre-experiment resamples (§4.2).

Arm	All (48)	Recoverable (11)	Ceiling (14)	Early origin (23)	Stuck + over-commit (24)
Continue	.34	.27	.00	.59	.08
Reset H	.27 (−.07 [−.15, +.00])	.18 (−.09 [−.27, +.11])	.00 (+.00 [+0.00, +.00])	.48 (−.11 [−.24, +.00])	.04 (−.04 [−.15, +.05])
Roll back W + reset H	.27 (−.07 [−.20, +.06])	.27 (+.00 [−.35, +.46])	.00 (+.00 [+0.00, +.00])	.43 (−.15 [−.33, −.02])	.04 (−.04 [−.17, +.10])
Roll back W, keep H	.31 (−.03 [−.15, +.08])	.27 (+.00 [−.50, +.40])	.00 (+.00 [+0.00, +.00])	.52 (−.07 [−.15, +.02])	.08 (+.00 [−.15, +.16])
Fresh R, keep H	.31 (−.03 [−.09, +.02])	.18 (−.09 [−.22, +.00])	.00 (+.00 [+0.00, +.00])	.57 (−.02 [−.11, +.07])	.04 (−.04 [−.11, +.00])
Reset H + fresh R	.25 (−.09 [−.19, +.00])	.18 (−.09 [−.27, +.11])	.00 (+.00 [+0.00, +.00])	.43 (−.15 [−.34, +.02])	.00 (−.08 [−.20, +.00])
Summary H	.26 (−.10 [−.19, −.01])	.20 (−.10 [−.29, +.12])	.00 (+.00 [+0.00, +.00])	.43 (−.15 [−.30, +.00])	.09 (+.00 [−.06, +.07])
Ledger H (full)	.29 (−.05 [−.15, +.03])	.18 (−.09 [−.27, +.11])	.00 (+.00 [+0.00, +.00])	.52 (−.07 [−.23, +.07])	.04 (−.04 [−.15, +.05])
Ledger − plans	.29 (−.05 [−.14, +.04])	.18 (−.09 [−.27, +.11])	.00 (+.00 [+0.00, +.00])	.52 (−.07 [−.23, +.09])	.04 (−.04 [−.15, +.05])
Ledger − reasoning	.17 (−.18 [−.32, −.05])	.00 (−.27 [−.59, −.04])	.00 (+.00 [+0.00, +.00])	.35 (−.24 [−.48, −.02])	.04 (−.04 [−.15, +.05])

Arm	Under-commit (22)	MettleBench (27)	SWE-smith (21)	GPT-6 Luna (19)	DeepSeek (29)
Continue	.66	.17	.57	.16	.47
Reset H	.55 (−.11 [−.24, +.00])	.11 (−.06 [−.15, +.04])	.48 (−.10 [−.24, +.02])	.11 (−.05 [−.18, +.00])	.38 (−.09 [−.19, +.02])
Roll back W + reset H	.50 (−.16 [−.34, −.02])	.11 (−.06 [−.25, +.17])	.48 (−.10 [−.24, +.02])	.21 (+.05 [−.18, +.30])	.31 (−.16 [−.28, −.03])
Roll back W, keep H	.59 (−.07 [−.20, +.07])	.19 (+.02 [−.19, +.20])	.48 (−.10 [−.17, −.02])	.16 (+.00 [+0.00, +.00])	.41 (−.05 [−.25, +.13])
Fresh R, keep H	.64 (−.02 [−.12, +.07])	.11 (−.06 [−.12, +.00])	.57 (+.00 [−.09, +.10])	.16 (+.00 [+0.00, +.00])	.41 (−.05 [−.14, +.03])
Reset H + fresh R	.55 (−.11 [−.31, +.07])	.07 (−.09 [−.24, +.02])	.48 (−.10 [−.25, +.07])	.11 (−.05 [−.18, +.00])	.34 (−.12 [−.27, +.02])
Summary H	.45 (−.20 [−.36, −.04])	.12 (−.06 [−.15, +.04])	.45 (−.15 [−.33, +.02])	.17 (+.00 [+0.00, +.00])	.32 (−.16 [−.29, −.02])
Ledger H (full)	.59 (−.07 [−.23, +.07])	.11 (−.06 [−.15, +.04])	.52 (−.05 [−.23, +.08])	.16 (+.00 [+0.00, +.00])	.38 (−.09 [−.23, +.04])
Ledger − plans	.59 (−.07 [−.23, +.10])	.11 (−.06 [−.15, +.04])	.52 (−.05 [−.23, +.11])	.11 (−.05 [−.18, +.00])	.41 (−.05 [−.18, +.07])
Ledger − reasoning	.32 (−.34 [−.57, −.11])	.00 (−.17 [−.37, −.02])	.38 (−.19 [−.40, +.05])	.11 (−.05 [−.18, +.00])	.21 (−.26 [−.47, −.05])

B State composition

Table 3: **Composition of the 48 states.** Recoverability classes come from the pre-experiment resamples. Types for originals are hindsight types, and for resampled states they are the judge’s types. One state had an unparseable type and one was untyped.

	Ceiling	Recoverable	Early origin
States (origins)	14 (7)	11 (6)	23 (12)
GPT-6 Luna / DeepSeek	10 / 4	6 / 5	3 / 20
MettleBench / SWE-smith	12 / 2	11 / 0	4 / 19
over-commit / stuck / under-commit	11 / 3 / 0	6 / 1 / 2	0 / 3 / 20
Mean start score	0.71	0.65	0.08
CONTINUE solve rate	0.00	0.27	0.59

C Arm availability and run accounting

Arms were marked unavailable by rules fixed before their outcomes; no substitute arm was ever run in their place.

- *Single-window states.* In 15 states the stall had only one implemented-decision window. All 15 are under-commit, 14 are DeepSeek and 12 are SWE-smith. Removing the current approach would leave an empty Ledger, so the six approach-boundary arms were recorded as equivalent to Reset H or as removing nothing, and were not run. This accounts for 90 cells.
- *Summaries.* No summary within the length tolerance was produced for 2 states.
- *Observation mask.* The mask could not be built for 27 states. Across 29 build attempts (including two second-repeat attempts), 22 failed because the delivery evidence of a retained tool message was ambiguous, and 7 because the source was a resumed run without a single natural run start.

- *Rollback donor*: For 4 resampled states, the *W* rollback checkpoint of the two combined arms lay in the origin run rather than the branch, which the runner refuses. This accounts for 8 build failures.

Of the 912 first-repeat cells, 784 were recorded. Of the second repeat, 105 cells were recorded: CONTINUE on all 48 states, Reset H on 13, the full Ledger on 14, and 2 for every other arm except the observation mask. The remaining 789 second-repeat cells were not run because we stopped the run. During the run, Docker overlay contention on the single host forced 32 in-flight attempts to be terminated and retried under fresh identifiers. Aborted attempts never count as outcomes. Four branches were lost and then retried and recorded: two to accounting interruptions, one to a transport error, and one to a timeout during a 20-minute operator pause. Twenty branches in flight at the final stop left no outcome.

D Second-opinion judge

The shadow judge is Claude Opus 5.5 given the frozen judge prompt and the same prefix-only record, with one vote per warning. It is capped to the warnings the primary judge evaluated, and it is record-only: its output was never read by state selection. It made 195 calls across the resample runs. Eight calls were refused by the model’s safeguards; each was re-issued with the worker’s reasoning blocks removed and then answered.

- Of the 92 runs where the primary judge confirmed a stall, the shadow judge chose the same primary stall in 70. In 8 it chose a different one, and in 14 it confirmed no stall. It confirmed a stall in 6 runs where the primary judge confirmed none.
- On the 70 shared primaries, the two judges agreed on type in 57 cases. The most common disagreement was under-commit relabelled as stuck (6 cases).
- Among the 23 resampled exp48 states, the shadow judge found no stall in 4, a different checkpoint in 2, and a different type in 3.